MODULE LINUX

Cours du 22 Janvier 2026

TP
Faire un script qui modifie le nom de la machine (fichiers hostname et hosts), les informations IP de la carte enp0s3 en fonction des données de l'utilisateur :

```
#!/bin/bash
#create by pascal
#date 21 janvier 2026
echo  "Veuillez saisir l'adresse IP :"
read ipv4
echo  "Veuillez saisir votre masque :"
read maskipv4
echo  "Veuillez saisir votre gateway :"
read passerelle

nomactuel=$HOSTNAME
echo  "Veuillez changer votre nom de PC :"
read nouveaunom


sudo sed -i.bak "/^address/caddress ${ipv4}" /etc/network/interfaces
sudo sed -i "/^netmask/cnetmask ${maskipv4}" /etc/network/interfaces
sudo sed -i "/^gateway/cgateway ${passerelle}" /etc/network/interfaces

sudo sed -i.bak "s/${nomactuel}/${nouveaunom}/g" /etc/hosts

sudo hostnamectl set-hostname ${nouveaunom}


sudo reboot
```

Methode 02

```
#!/bin/bash
#create by pascal
#date 21 janvier 2026
echo  "Veuillez saisir l'adresse IP :"
read ipv4
echo  "Veuillez saisir votre masque :"
read maskipv4
echo  "Veuillez saisir votre gateway :"
read passerelle

nomactuel=$HOSTNAME
echo  "Veuillez changer votre nom de PC :"
read nouveaunom

cp /etc/network/interfaces /etc/network/interfaces.bak
cat > /etc/network/interfaces << ENVOILACONFIG

# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
#allow-hotplug enp0s3
#iface enp0s3 inet dhcp
allow-hotplug enp0s3
iface enp0s3 inet static
address ${ipv4}
netmask ${maskipv4}
gateway ${passerelle}

ENVOILACONFIG

sed -i.bak "s/${nomactuel}/${nouveaunom}/g" /etc/hosts
hostnamectl set-hostname ${nouveaunom}


reboot
```

cat n'aime pas être associé avec la commande sudo dans le script
donc pour lancer un script

```
sudo bash maconfig.sh
```

```
#create by pascal
#date 21 janvier 2026
memoire=$(free -h | grep Mem | cut -d' ' -f 12)
hd=$(df -h | grep '^/dev' | cut -d' ' -f 12)
nbcoeur=$(cat /proc/cpuinfo | grep 'cpu cores' | cut -d':' -f 2)
usersudo=$(cat /etc/group | grep '^sudo' | cut -d':' -f4)

echo "LA TAILLE DE LA MEMOIRE : ${memoire}"
echo "LA TAILLE DE SDA : ${hd}"
echo "LE NB DE COEUR : ${nbcoeur}"
echo "MES USERS SUDO : ${usersudo}"
```

```
COMMANDE AWK => On l'utilise quand la commande cut n'est pas satisfaisante
Print => afficher à l'ecran
$0 => la ligne entier
$1 => le premier champ
$2 => le deuxieme champ
$3 => le troisieme champ
…
gsub => pour remplacer une chaine par une autre chaine
-F => pour définir le délimiteur
```

```
ip -br a | grep enp0s3 | cut -d' ' -f 25
192.168.240.201/24

ip -br a | grep enp0s3
enp0s3           UP             192.168.240.201/24
fe80::a00:27ff:fe7d:5220/64

ip -br a | grep enp0s3 | awk '{print $0}'
enp0s3           UP             192.168.240.201/24
fe80::a00:27ff:fe7d:5220/64

ip -br a | grep enp0s3 | awk '{print $1}'
enp0s3

ip -br a | grep enp0s3 | awk '{print $2}'
UP

ip -br a | grep enp0s3 | awk '{print $3}'
192.168.240.201/24

ip -br a | grep enp0s3 | awk '{print $3}' | cut -d'/' -f 1
192.168.240.201

ip -br a | grep enp0s3 | awk '{print $3}' | cut -d'/' -f 2
```

24

```
ip -br a | grep enp0s3 | awk '{gsub(/24/,"");print $3}'
192.168.0.201/

ip -br a | grep enp0s3 | awk '{gsub(//24/,"");print $3}'
awk: line 1: syntax error at or near ,

ip -br a | grep enp0s3 | awk '{gsub(/\/24/,"");print $3}'
192.168.240.201
```

Exemple:

```
free -h | grep Mem | cut -d' ' -f 12
free -h | grep Mem | awk '{print $2}'

df -h | grep '^/dev' | cut -d' ' -f 12
df -h | grep '^/dev' | awk '{print $2}'

cat /proc/cpuinfo | grep 'cpu cores' | cut -d':' -f 2
cat /proc/cpuinfo | grep 'cpu cores' | awk '{print $4}'

cat /etc/group | grep '^sudo' | cut -d':' -f4
cat /etc/group | grep '^sudo' | awk -F: '{print $4}'
```

LES TABLEAUX

```
#!/bin/bash
#create by pascal
#date 22  Janvier 26
inforeseau=(192.168.240.201 255.255.255.0 192.168.240.254)
echo "tous les elements ${inforeseau[*]}"
echo "Pour afficher l adresse ip ${inforeseau[0]}"
echo "Pour afficher le mask ${inforeseau[1]}"
echo "Pour afficher la passerelle ${inforeseau[2]}"
echo "pour avoir le nombre d elements ${#inforeseau[*]}"
```

LES CONDIDITIONS

Syntaxe

```
if [[ votre test1 ]]
then
      Votre commande1
elif [[ votre test2 ]]
then
      Votre commande2
else
      Votre commandeN
Fi
```

Opérateur

= ou (-eq)	=> égale
!= ou(-ne)	=> diffèrent
> ou (-gt)	=> supérieur
< ou (-lt)	=> inferieur
>= ou (-ge)	=> supérieur ou égale
<= ou (-le)	=> Inferieur ou égale

Pour faire un **ou** dans condition utiliser **||**

```
#!/bin/bash
#create by pascal
#date 22  Janvier 26
echo  "Etes vous heureux : (oui/non)"
read reponse

if [[ ${reponse} = 'oui' ]]
then
      echo "super nouvelle !!!!"
elif [[ ${reponse} = 'non' ]]
then
      echo "Mangez du chocolat !!"
else
      echo "Je n'ai pas compris votre reponse"
fi
```

version 2

```
#!/bin/bash
#create by pascal
#date 22  Janvier 26
echo  "Etes vous heureux : ((o)ui/(n)on)"
read reponse

if [[ ${reponse} = 'oui' ]] || [[ ${reponse} = 'o' ]]
then
      echo "super nouvelle !!!!"
elif [[ ${reponse} = 'non' ]] || [[ ${reponse} = 'n' ]]
then
      echo "Mangez du chocolat !!"
else
      echo "Je n'ai pas compris votre reponse"
fi
```

version 3
^oui$ ^o$

```
#!/bin/bash
#create by pascal
#date 22  Janvier 26
echo  "Etes vous heureux : ((o)ui/(n)on)"
read reponse

if [[ ${reponse} =~ ^(oui|o)$ ]]
then
      echo "super nouvelle !!!!"
elif [[ ${reponse} =~ ^(non|n)$ ]]
then
      echo "Mangez du chocolat !!"
else
      echo "Je n'ai pas compris votre reponse"
fi
```

version 4 => Pour Debian

Il existe une commande pour activer ou desactiver des options dans notre shell => shopt
Pour activer un option on utilise le paramètre -s
Pour désactiver un option on utilise le paramètre -u

```
#!/bin/bash
#create by pascal
#date 22  Janvier 26

shopt -s nocasematch

echo  "Etes vous heureux : ((o)ui/(n)on)"
read reponse

if [[ ${reponse} =~ ^(oui|o)$ ]]
then
      echo "super nouvelle !!!!"
elif [[ ${reponse} =~ ^(non|n)$ ]]
then
      echo "Mangez du chocolat !!"
else
      echo "Je n'ai pas compris votre reponse"
fi
```

TP Amélioration du Script
O OUI Y YES
Pour avoir la reponse Super nouvelle

N NON NO
Pour avoir la reponse Mangez du chocolat

Solution 01

```
#!/bin/bash
#create by pascal
#date 22  Janvier 26

shopt -s nocasematch
echo  "Etes vous heureux : ((o)ui |(y)es /(n)on | no)"
read reponse

if [[ ${reponse} = 'oui' ]] || [[ ${reponse} = 'o' ]] || [[ ${reponse} =
'y' ]] || [[ ${reponse} = 'yes' ]]
then
      echo "super nouvelle !!!!"

elif [[ ${reponse} = 'non' ]] || [[ ${reponse} = 'n' ]] || [[ ${reponse} =
'no' ]]
then
      echo "Mangez du chocolat !!"
else
      echo "Je n'ai pas compris votre reponse"
fi
```

```
#!/bin/bash
#create by pascal
#date 22  Janvier 26

shopt -s nocasematch

echo  "Etes vous heureux : ((o)ui |(y)es /(n)on | no)"
read reponse

if [[ ${reponse} =~ ^(oui|o|yes|y)$ ]]
then
      echo "super nouvelle !!!!"

elif [[ ${reponse} =~ ^(non|no|n)$ ]]
then
      echo "Mangez du chocolat !!"
else
      echo "Je n'ai pas compris votre reponse"
fi
```

TESTER LA PRESENCE D UN FICHIER

METHODE 01

```
#!/bin/bash
#create by pascal
#date 22  Janvier 26
fichier=/home/tssr/scripts/tssr.txt

if [[ -f ${fichier} ]]
then
      echo "Le fichier ${fichier} existe"
else
      echo "Le fichier ${fichier} n'existe PAS "
fi
```

METHODE 02 – Par la négation => !

```
#!/bin/bash
#create by pascal
#date 22  Janvier 26
fichier=/home/tssr/scripts/tssr.txt

if ! [[ -f ${fichier} ]]
then
      echo "Le fichier ${fichier} n'existe PAS"
else
      echo "Le fichier ${fichier} existe"
fi
```

METHODE 03

```
#!/bin/bash
#create by pascal
#date 22  Janvier 26
fichier=/home/tssr/scripts/tssr.txt

if [[ -f ${fichier} ]]
then
      echo "Le fichier ${fichier} existe"
else
      echo "Le fichier ${fichier} n'existe PAS "
      touch ${fichier}
fi
```

LES BOUCLE WHILE

```
while [ instruction 1 ]
do
Instruction 2
Instruction 3
Done
```

Opérateur

= ou (-eq)	=> égale
!= ou(-ne)	=> diffèrent
> ou (-gt)	=> supérieur
< ou (-lt)	=> inferieur
>= ou (-ge)	=> supérieur ou égale
<= ou (-le)	=> Inferieur ou égale

Execution des instruction 2 et 3 tant que l'instruction 1 est VRAI

```
OPTIONS
-f fichier => vrai si fichier existe
-d nomdossier => vrai si dossier existe
-x fichier => vrai si fichier autorisé en execution
-w fichier => vrai si fichier autorisé en écriture
-r fichier => vrai si fichier autorisé en lecture
-s fichier => vrai si le fichier n'est pas vide
```

Methode 01

```
#!/bin/bash
#create by pascal
#date 22  Janvier 26
shopt -s nocasematch

echo  "Etes vous heureux : ((o)ui |(y)es /(n)on | no)"
read reponse

var=0

while [ ${var} = 0 ]
do

      if [[ ${reponse} =~ ^(oui|o|yes|y)$ ]]
      then
            echo "super nouvelle !!!!"
            var=1
      elif [[ ${reponse} =~ ^(non|no|n)$ ]]
      then
            echo "Mangez du chocolat !!"
            var=1
```

```
      else
            echo "Je n'ai pas compris votre reponse"
            echo  "Etes vous heureux : ((o)ui |(y)es /(n)on | no)"
            read reponse

      fi

done

echo fin de programme
```

Methode 02 – break permet de quitter une boucle

```
#!/bin/bash
#create by pascal
#date 22  Janvier 26

shopt -s nocasematch

echo  "Etes vous heureux : ((o)ui |(y)es /(n)on | no)"
read reponse

var=0

while [ ${var} = 0 ]
do

      if [[ ${reponse} =~ ^(oui|o|yes|y)$ ]]
      then
            echo "super nouvelle !!!!"
            break;
      elif [[ ${reponse} =~ ^(non|no|n)$ ]]
      then
            echo "Mangez du chocolat !!"
            break;
      else
            echo "Je n'ai pas compris votre reponse"
            echo  "Etes vous heureux : ((o)ui |(y)es /(n)on | no)"
            read reponse

      fi


done

echo fin de programme
```

Ici LE SCRIPT BOUCLE TANT QUE LE FICHIER tssr.txt n'est pas présent

```
#!/bin/bash
```

```
#create by pascal
#date 22  Janvier 26
fichier=/home/tssr/scripts/tssr.txt

while ! [[ -f ${fichier} ]]
do
      sleep 10
done

echo "le fichier existe"
```

Pour lancer un script en fond de tache , il faut ajouter & apres le nom du script

```
bash s17.sh &
touch tssr.txt
```

CASE

Syntaxe

```
case mot in
Cas numero 1)
Instruction 1
;;
Cas numero 2)
Instruction 2
;;
Cas numero 3)
Instruction 3
;;
Pour les autres cas)
Instruction
;;
esac
```

```
#!/bin/bash
#create by pascal
#date 21 janvier 2026

echo  "1 – Affiche la date"
echo  "2 – Affiche mon IP"
echo  "3 – Affiche la version de Debian"

echo "Veuillez choisir votre menu :"
read menu

case ${menu} in
1)
date
;;
2)
```

```
ip -br a | grep enp0s3 | awk '{print $3}' | cut -d'/' -f 1
;;
3)
cat /etc/debian_version
;;
*)
echo "Choix incorrecte"
;;
esac

#!/bin/bash
#create by pascal
#date 21 janvier 2026

echo  "1 – Affiche la date"
echo  "2 – Affiche mon IP"
echo  "3 – Affiche la version de Debian"

echo "Veuillez choisir votre menu :"
read menu

var=0

while [ ${var} = 0 ]
do


      case ${menu} in
      1)
            Date
            var=1
            ;;
      2)
            ip -br a | grep enp0s3 | awk '{print $3}' | cut -d'/' -f 1
            var=1
            ;;
      3)
            cat /etc/debian_version
            var=1
            ;;
      *)
            echo "Choix incorrecte"
            echo "Veuillez choisir votre menu :"
            read menu

            ;;
      esac

done
```

Version Babacar

```
#!/bin/bash
#create by Yinshi
#date 21 janvier 2026
while [ 1 ]
do
      echo
      echo "1 – Affiche la date"
      echo "2 – Affiche mon IP"
      echo "3 – Affiche la version de Debian"
      echo "4 - exit"
      echo
      echo "Veuillez choisir votre menu :"
      read menu
      case ${menu} in
            1)
                  date
                  ;;
            2)
                  ip -br a | grep enp0s3 | awk '{print $3}' | cut -d'/' -f
            1
                  ;;
            3)
                  cat /etc/debian_version
                  ;;
            4)
                  exit
                  ;;
            *)
                  echo "Choix incorrecte"
                  ;;
      esac
done
```

LES CALCULES MATHEMATIQUE

```
tssr@lx02:~/scripts$ nombre=9
tssr@lx02:~/scripts$ echo ${nombre}
```

```
9
tssr@lx02:~/scripts$ let nombre+=1
tssr@lx02:~/scripts$ echo ${nombre}
10
tssr@lx02:~/scripts$ let nombre-=1
tssr@lx02:~/scripts$ echo ${nombre}
9
tssr@lx02:~/scripts$ let nombre+=3
tssr@lx02:~/scripts$ echo ${nombre}
12
tssr@lx02:~/scripts$ let nombre2=nombre+5
tssr@lx02:~/scripts$ echo ${nombre}
12
tssr@lx02:~/scripts$ echo ${nombre2}
17
tssr@lx02:~/scripts$ nombre2=$((nombre2+3))
tssr@lx02:~/scripts$ echo ${nombre2}
20
tssr@lx02:~/scripts$
```

Afficher 10 fois bonjour

```
#!/bin/bash
#create by pascal
#date 21 janvier 2026
var=0
while [ ${var} -lt 10 ]
do
        let var+=1
        #var=$((var+1))
        echo "${var} - Bonjour"

done
```

LA BOUCLE FOR

```
#!/bin/bash
#create by pascal
#date 21 janvier 2026
echo "METHODE 01"

for var in 1 2 3 4 5 6 7 8 9 10
do
      echo "${var} - Bonjour"
done

echo "METHODE 02"

for var in {1..10}
do
      echo "${var} - Bonjour"
done

echo "METHODE 03"

for (( var=1; var<=10 ; var++ ))
do
      echo "${var} - Bonjour"
done


echo "METHODE 04"

for (( var=1; var<=10 ; var+=2 ))
do
      echo "${var} - Bonjour"
done
```

LE GENIE CAR LINUX NOUS REND GENIAL

```
tab=({0..1}{0..1})

echo ${tab[*]}
00 01 10 11
```

Convertisseur en binaire

```
tab=({0..1}{0..1}{0..1}{0..1}{0..1}{0..1}{0..1}{0..1})

echo ${tab[44]}
00101100

echo ${tab[100]}
01100100
```

```
echo ${tab[92]}
01011100

echo ${tab[*]}
00000000 00000001 00000010 00000011 00000100 00000101 00000110 00000111 00001000 00001001 00001010
00001011 00001100 00001101 00001110 00001111 00010000 00010001 00010010 00010011 00010100 00010101
00010110 00010111 00011000 00011001 00011010 00011011 00011100 00011101 00011110 00011111 00100000
00100001 00100010 00100011 00100100 00100101 00100110 00100111 00101000 00101001 00101010 00101011
00101100 00101101 00101110 00101111 00110000 00110001 00110010 00110011 00110100 00110101 00110110
00110111 00111000 00111001 00111010 00111011 00111100 00111101 00111110 00111111 01000000 01000001
01000010 01000011 01000100 01000101 01000110 01000111 01001000 01001001 01001010 01001011 01001100
01001101 01001110 01001111 01010000 01010001 01010010 01010011 01010100 01010101 01010110 01010111
01011000 01011001 01011010 01011011 01011100 01011101 01011110 01011111 01100000 01100001 01100010
01100011 01100100 01100101 01100110 01100111 01101000 01101001 01101010 01101011 01101100 01101101
01101110 01101111 01110000 01110001 01110010 01110011 01110100 01110101 01110110 01110111 01111000
01111001 01111010 01111011 01111100 01111101 01111110 01111111 10000000 10000001 10000010 10000011
10000100 10000101 10000110 10000111 10001000 10001001 10001010 10001011 10001100 10001101 10001110
10001111 10010000 10010001 10010010 10010011 10010100 10010101 10010110 10010111 10011000 10011001
10011010 10011011 10011100 10011101 10011110 10011111 10100000 10100001 10100010 10100011 10100100
10100101 10100110 10100111 10101000 10101001 10101010 10101011 10101100 10101101 10101110 10101111
10110000 10110001 10110010 10110011 10110100 10110101 10110110 10110111 10111000 10111001 10111010
10111011 10111100 10111101 10111110 10111111 11000000 11000001 11000010 11000011 11000100 11000101
11000110 11000111 11001000 11001001 11001010 11001011 11001100 11001101 11001110 11001111 11010000
11010001 11010010 11010011 11010100 11010101 11010110 11010111 11011000 11011001 11011010 11011011
11011100 11011101 11011110 11011111 11100000 11100001 11100010 11100011 11100100 11100101 11100110
11100111 11101000 11101001 11101010 11101011 11101100 11101101 11101110 11101111 11110000 11110001
11110010 11110011 11110100 11110101 11110110 11110111 11111000 11111001 11111010 11111011 11111100
11111101 11111110 11111111
```

```
#!/bin/bash
#create by pascal
#date 21 janvier 2026

echo "METHODE 01"

for ligne in ls *.sh
do
echo ${ligne}
done


echo "METHODE 02" - CONSEILLE

for ligne in $(ls *.sh)
do
echo ${ligne}
done
```

```
#!/bin/bash
#create by pascal
#date 21 janvier 2026

echo "METHODE 02" - CONSEILLE

var=0
for ligne in $( cat /etc/passwd | cut -d':' -f 1)
do
let var+=1
echo "User ${var} = ${ligne}"
done
```

GESTION DES ERREURS
La variable $? Renvoi le resultat d'une commande
Si $? = 0 Pas d'erreur
Si $? Renvoi autre chose que 0 , alors on une erreur

```
tssr@lx02:~/scripts$ ls *.sh
maconfig2.sh  s11.sh  s14.sh  s17.sh  s20.sh  s23.sh
s09.sh        s12.sh  s15.sh  s18.sh  s21.sh  s24.sh
s10.sh        s13.sh  s16.sh  s19.sh  s22.sh
```

```
tssr@lx02:~/scripts$ ls *.sh > resultat.txt
```

```
tssr@lx02:~/scripts$ ls
maconfig2.sh  s10.sh  s13.sh  s16.sh  s19.sh  s22.sh  tssr.txt
resultat.txt  s11.sh  s14.sh  s17.sh  s20.sh  s23.sh
s09.sh        s12.sh  s15.sh  s18.sh  s21.sh  s24.sh
```

```
tssr@lx02:~/scripts$ cat resultat.txt
maconfig2.sh
s09.sh
s10.sh
s11.sh
s12.sh
s13.sh
s14.sh
s15.sh
s16.sh
s17.sh
s18.sh
s19.sh
s20.sh
s21.sh
s22.sh
s23.sh
s24.sh
```

```
tssr@lx02:~/scripts$ ls
maconfig2.sh  s11.sh  s14.sh  s17.sh  s20.sh  s23.sh
s09.sh        s12.sh  s15.sh  s18.sh  s21.sh  s24.sh
s10.sh        s13.sh  s16.sh  s19.sh  s22.sh  tssr.txt
```

```
tssr@lx02:~/scripts$ ls *.sh > resultat.txt 2>error.txt
tssr@lx02:~/scripts$ ls
error.txt     s09.sh  s12.sh  s15.sh  s18.sh  s21.sh  s24.sh
maconfig2.sh  s10.sh  s13.sh  s16.sh  s19.sh  s22.sh  tssr.txt
resultat.txt  s11.sh  s14.sh  s17.sh  s20.sh  s23.sh
```

```
tssr@lx02:~/scripts$ cat resultat.txt
```

```
maconfig2.sh
s09.sh
s10.sh
s11.sh
s12.sh
s13.sh
s14.sh
s15.sh
s16.sh
s17.sh
s18.sh
s19.sh
s20.sh
s21.sh
s22.sh
s23.sh
s24.sh
```

```
tssr@lx02:~/scripts$ cat error.txt
```

```
tssr@lx02:~/scripts$ ls
maconfig2.sh  s11.sh  s14.sh  s17.sh  s20.sh  s23.sh
s09.sh        s12.sh  s15.sh  s18.sh  s21.sh  s24.sh
s10.sh        s13.sh  s16.sh  s19.sh  s22.sh  tssr.txt
```

```
tssr@lx02:~/scripts$ ls -125566666 *.sh > resultat.txt 2>error.txt
tssr@lx02:~/scripts$ ls
error.txt     s09.sh  s12.sh  s15.sh  s18.sh  s21.sh  s24.sh
maconfig2.sh  s10.sh  s13.sh  s16.sh  s19.sh  s22.sh  tssr.txt
resultat.txt  s11.sh  s14.sh  s17.sh  s20.sh  s23.sh
```

```
tssr@lx02:~/scripts$ cat resultat.txt
tssr@lx02:~/scripts$ cat error.txt
ls : option invalide -- '2'
Saisissez « ls --help » pour plus d'informations.
```

Script

Fichier **users.txt**

Ajouter ses lignes dans 1 fichier users.txt

alex:sudo:Reponse2025
alain:vide:Reponse2025
samantha:vide:Reponse2025
romeo:sudo:Reponse2025
claire:vide:Reponse2025

#1- VERIFIER QUE LE FICHIER users.txt EXISTE (sinon on quitte le script)
#2- ON DEMANDE A L UTILISATEUR DU SCRIPT S IL VEUT IMPORTER LES 5 USERS
(si touche o , on continue , sinon on quitte le script)
#3- A L AIDE DE LA BOUCLE FOR , ON SCAN LE CONTENU DE users.txt ET ON CREE LES UTILISATEURS DANS LINUX

AIDE
~~adduser =>Commande interactive NON UTILE POUR NOUS~~

useradd => Commande non interactive (parfait pour un script)
chpasswd => Modifier le mot de passe d'un user
userdel -r => Supprimer un utilisateur et son répertoire d'origine
usermod => Ajouter un utilisateur dans un groupe

Exemple
useradd pascal --home-dir /home/pascal/ --create-home
chpasswd pascal:Reponse2025
userdel -r pascal
usermod -aG sudo pascal

AVEC GESTION DES ERREUR

```
#!/bin/bash
#create by pascal
#date 21 janvier 2026

echo "######## E T A P E    0 1 ############"
echo " VERIFIER QUE LE FICHIER users.txt EXISTE (sinon on quitte le script)"
fichier=/home/tssr/scripts/users.txt

if ! [[ -f ${fichier} ]]
then
        echo "JE PEUX RIEN FAIRE POUR TOI"
        echo "RELANCE MOI QUAND TU AURAS CREER LE FICHIER"
        exit
fi
#####################################################################
echo "######## E T A P E    0 2 ############"
echo " ON DEMANDE A L UTILISATEUR DU SCRIPT S IL VEUT IMPORTER LES 5 USERS"
echo "Veux tu importer les users (o/n)"
read reponse

if [[ ${reponse} = "o" ]]
then
        echo "######## E T A P E    0 3 ############"
        echo " ON IMPORTE LES 5 USERS"
        for ligne in $(cat ${fichier} )
        do
                login=$(echo ${ligne} | cut -d':' -f 1)
                nomgroupe=$(echo ${ligne} | cut -d':' -f 2)
                motdepasse=$(echo ${ligne} | cut -d':' -f 3)
                echo  "le login est ${login}"
                echo  "le groupe est ${nomgroupe}"
                echo  "password est ${motdepasse}"


                sudo useradd ${login} --home-dir /home/${login}/ --create-home

if [[ $? -ne 0 ]]
then
        echo "ECHEC POUR LA CREATION DE ${login}"
exit
else
        echo ${login}:${motdepasse} | sudo chpasswd
        if [[ $? -ne 0 ]]
        then
                echo "ECHEC POUR le MOT DE PASSE DE ${login}"
                exit
        else
                if [[ ${nomgroupe} = 'sudo' ]]
                then
                        sudo usermod -aG sudo ${login}
                fi
        fi
fi
        done
else
        echo "BAH C ETAIT RAPIDE CAR TU NE VEUX PAS IMPORTER"
        echo "Bonne soirée"
        exit
fi
```

SANS GESTION DES ERREUR – PLUS SIMPLE

```
#!/bin/bash
#create by pascal
#date 21 janvier 2026

echo "######## E T A P E    0 1 ############"
echo " VERIFIER QUE LE FICHIER users.txt EXISTE (sinon on quitte le script)"
fichier=/home/tssr/scripts/users.txt

if ! [[ -f ${fichier} ]]
then
       echo "JE PEUX RIEN FAIRE POUR TOI"
       echo "RELANCE MOI QUAND TU AURAS CREER LE FICHIER"
       exit
fi
#####################################################################
echo "######## E T A P E    0 2 ############"
echo " ON DEMANDE A L UTILISATEUR DU SCRIPT S IL VEUT IMPORTER LES 5 USERS"
echo "Veux tu importer les users (o/n)"
read reponse

if [[ ${reponse} = "o" ]]
then
       echo "######## E T A P E    0 3 ############"
       echo " ON IMPORTE LES 5 USERS"
       for ligne in $(cat ${fichier} )
       do
              login=$(echo ${ligne} | cut -d':' -f 1)
              nomgroupe=$(echo ${ligne} | cut -d':' -f 2)
              motdepasse=$(echo ${ligne} | cut -d':' -f 3)

              sudo useradd ${login} --home-dir /home/${login}/ --create-home

              echo ${login}:${motdepasse} | sudo chpasswd

              if [[ ${nomgroupe} = 'sudo' ]]
              then
                     sudo usermod -aG sudo ${login}
              fi


       done
else
       echo "BAH C ETAIT RAPIDE CAR TU NE VEUX PAS IMPORTER"
       echo "Bonne soirée"
fi
```